

Rapport de stage

Développement d'un portail web institutionnel

Projet : Portail Ministère

Stagiaire : Wassim Elouafik

Filière : Développement Digital, 2ème année

Établissement : Institut Spécialisé de Technologie Appliquée Sidi Moumen Casablanca

Entreprise / organisme d'accueil : Save Environment

Service d'accueil : Service informatique

Ville : Casablanca

Encadrant pédagogique : Yassine Dabachine

Encadrant professionnel : Azzedine Elouafik

Période du stage : du 13/04/2026 au 13/05/2026

Année universitaire : 2025-2026

Remerciements

Le stagiaire tient à exprimer ses sincères remerciements à toutes les personnes qui ont contribué au bon déroulement du stage et à la réalisation de ce projet. Des remerciements particuliers sont adressés à l'encadrant professionnel, Monsieur Azzedine Elouafik, pour son accompagnement, ses conseils et ses orientations tout au long de la période de stage.

Des remerciements sont également adressés à l'encadrant pédagogique, Monsieur Yassine Dabachine, pour son suivi, ainsi qu'à l'ensemble des membres de Save Environment et du service informatique pour leur disponibilité, leur aide et leur confiance. Cette expérience a permis de mettre en pratique les connaissances techniques acquises à l'Institut Spécialisé de Technologie Appliquée Sidi Moumen Casablanca, de développer de nouvelles compétences et de mieux comprendre les exigences d'un projet web professionnel.

Résumé

Ce rapport présente le travail réalisé dans le cadre du stage effectué au sein de Save Environment, dans le service informatique, du 13 avril 2026 au 13 mai 2026. Le projet porte sur la conception et le développement d'un portail web institutionnel pour un ministère. Il vise à mettre en place une plateforme moderne permettant de présenter les actualités, les programmes, les publications, les statistiques et les informations générales de l'institution.

L'application est organisée autour d'une architecture full-stack. La partie frontend est développée avec React, Vite et Tailwind CSS, tandis que la partie backend repose sur Laravel et Sanctum pour la gestion sécurisée de l'authentification. Le portail comprend une interface publique destinée aux visiteurs et une interface d'administration permettant la gestion du contenu.

Le travail réalisé couvre l'analyse des besoins, la conception de l'architecture, le développement des interfaces, l'intégration des API, la gestion des données et la mise en place d'une base de projet évolutive. Ce stage a permis de renforcer les compétences du stagiaire en développement web moderne, en structuration de projet, en communication avec une API REST et en gestion d'un espace administrateur sécurisé.

Mots-clés : React, Laravel, API REST, Sanctum, Tailwind CSS, portail institutionnel, administration web.

Introduction générale

La transformation numérique occupe aujourd'hui une place importante dans le fonctionnement des administrations et des organismes publics. Les citoyens, les partenaires et les différents acteurs institutionnels ont besoin d'accéder rapidement à des informations fiables, structurées et régulièrement mises à jour. Dans ce contexte, les portails web institutionnels jouent un rôle essentiel dans la communication, la transparence et la diffusion de l'information.

Le projet réalisé pendant ce stage consiste à développer un portail web institutionnel pour un ministère, dans le cadre de la formation Développement Digital, 2ème année. L'objectif principal est de créer une plateforme claire, moderne et ergonomique qui permet de consulter les actualités, les programmes, les publications, les statistiques et les informations relatives à l'institution. Le projet prévoit aussi une interface d'administration permettant aux responsables de gérer les contenus sans intervenir directement dans le code source.

Ce rapport décrit les différentes étapes du projet, depuis l'analyse des besoins jusqu'à la réalisation technique. Il présente l'environnement de travail, les choix technologiques, l'architecture de l'application, les fonctionnalités développées, les difficultés rencontrées ainsi que les compétences acquises durant le stage.

1. Présentation de l'organisme d'accueil

1.1 Présentation générale

Le stage a été effectué au sein de Save Environment, un organisme situé à Casablanca. L'organisme intervient dans un contexte où les solutions numériques occupent une place importante dans l'organisation du travail, la communication et la gestion des informations.

Dans ce cadre, le développement d'un portail web institutionnel permet d'améliorer la visibilité des actions, de centraliser les informations importantes et de faciliter l'accès aux ressources publiques. Le projet réalisé répond donc à un besoin concret de structuration et de modernisation de la communication numérique.

1.2 Service d'accueil

Le stage s'est déroulé au sein du service informatique de Save Environment.

Le service d'accueil avait pour rôle de suivre l'avancement du projet, de préciser les besoins fonctionnels et de valider les choix réalisés. Cette collaboration a permis d'adapter la solution aux attentes de l'organisme et aux besoins des futurs utilisateurs.

1.3 Objectifs du stage

Les objectifs principaux du stage étaient les suivants :

- analyser le besoin fonctionnel d'un portail web institutionnel ;
- concevoir une architecture web moderne séparant le frontend et le backend ;
- développer une interface publique responsive et facile à utiliser ;
- mettre en place une API REST pour fournir les données au frontend ;
- développer un espace d'administration sécurisé ;
- structurer le projet afin de faciliter sa maintenance et son évolution.

2. Contexte et problématique du projet

2.1 Contexte

Les institutions publiques ont besoin d'outils numériques capables de diffuser rapidement leurs informations officielles. Un portail web permet de regrouper plusieurs types de contenus : actualités, programmes, documents, statistiques, contacts et présentation de l'institution.

Le projet répond donc à un besoin de modernisation de la communication institutionnelle. L'application doit être simple à consulter pour les visiteurs, mais aussi facile à administrer par les responsables internes.

2.2 Problématique

La problématique principale peut être formulée ainsi :

Comment concevoir et développer un portail web institutionnel moderne, sécurisé, responsive et facilement administrable ?

Pour répondre à cette problématique, plusieurs contraintes doivent être prises en compte :

- proposer une navigation claire pour le public ;
- organiser les contenus par catégories ;
- permettre la consultation détaillée des actualités et programmes ;
- rendre les publications accessibles ;
- intégrer un formulaire de contact ;
- protéger l'interface d'administration ;
- assurer une communication fiable entre frontend et backend ;
- préparer le projet pour de futures évolutions.

3. Analyse des besoins

3.1 Besoins fonctionnels

L'application développée doit permettre aux visiteurs de :

- consulter la page d'accueil du portail ;
- lire les actualités publiées par l'institution ;
- consulter les détails d'une actualité ;
- découvrir les programmes proposés ;
- consulter les détails d'un programme ;
- accéder aux publications et documents ;
- envoyer un message via le formulaire de contact ;
- consulter une page de présentation du ministère ;
- utiliser le site sur différents supports : ordinateur, tablette et mobile.

L'espace administrateur doit permettre aux utilisateurs autorisés de :

- se connecter à l'interface d'administration ;
- accéder à un tableau de bord ;
- gérer les actualités ;
- gérer les programmes ;
- gérer les publications ;
- consulter et modifier les statistiques ;
- se déconnecter de manière sécurisée.

3.2 Besoins non fonctionnels

En plus des fonctionnalités visibles, le projet doit respecter plusieurs exigences non fonctionnelles :

- Ergonomie : interface claire, moderne et simple à comprendre.
- Responsive design : adaptation aux différentes tailles d'écran.

- Sécurité : protection des routes d'administration.
- Maintenabilité : organisation du code en composants, pages, contextes et contrôleurs.
- Évolutivité : possibilité d'ajouter de nouvelles rubriques ou fonctionnalités.
- Performance : chargement optimisé grâce à Vite et au lazy loading des pages React.
- Accessibilité : navigation compréhensible et contenus bien structurés.

4. Environnement technique

4.1 Technologies frontend

La partie frontend du projet utilise les technologies suivantes :

- React 18 : bibliothèque JavaScript utilisée pour construire les interfaces utilisateur sous forme de composants.
- Vite 5 : outil de développement rapide permettant de lancer, compiler et prévisualiser l'application.
- Tailwind CSS 3 : framework CSS utilitaire utilisé pour construire une interface responsive et cohérente.
- React Router DOM : bibliothèque de routage permettant de gérer les pages publiques et les pages d'administration.
- Axios : client HTTP utilisé pour communiquer avec l'API Laravel.
- Framer Motion : bibliothèque utilisée pour améliorer l'expérience utilisateur avec des animations.
- Lucide React : bibliothèque d'icônes utilisée dans l'interface.
- React Helmet Async : outil pour gérer les métadonnées des pages.
- i18next / react-i18next : base prévue pour l'internationalisation.

4.2 Technologies backend

La partie backend repose sur :

- Laravel : framework PHP utilisé pour développer l'API REST.
- Laravel Sanctum : système d'authentification utilisé pour sécuriser l'espace administrateur.
- Eloquent ORM : outil de Laravel pour interagir avec la base de données à travers des modèles.
- Migrations Laravel : mécanisme permettant de créer et maintenir la structure de la base de données.
- Seeders : scripts permettant d'insérer des données initiales de test.
- PHPUnit : outil de test intégré à l'environnement Laravel.

4.3 Outils de développement

Les outils utilisés dans le projet incluent :

- Git et GitHub pour la gestion de version ;
- Visual Studio Code ou un éditeur similaire pour le développement ;
- Node.js et npm pour la partie frontend ;
- Composer pour la gestion des dépendances PHP ;

- Postman ou un navigateur pour tester les endpoints API ;
- un terminal pour lancer les serveurs de développement.

5. Architecture générale du projet

5.1 Organisation globale

Le projet est organisé en deux grandes parties :

- frontend/ : application React utilisée par les visiteurs et les administrateurs ;
- backend/ : application Laravel exposant les endpoints API et gérant les données.

Cette séparation permet de mieux organiser le travail. Le frontend se concentre sur l'expérience utilisateur, tandis que le backend prend en charge les données, la sécurité et les règles métier.

5.2 Architecture frontend

Le frontend est structuré autour des dossiers suivants :

- pages/ : contient les pages principales de l'application, comme l'accueil, les actualités, les programmes, les publications, le contact et l'administration.
- components/ : contient les éléments réutilisables, comme la barre de navigation, le footer, les boutons, les badges, les sections et les composants de l'administration.
- context/ : contient les contextes globaux, notamment l'authentification et la gestion de la langue.
- hooks/ : contient des hooks personnalisés pour les appels API, les animations au scroll et les effets de survol.
- utils/ : contient la configuration de communication avec l'API.

L'application utilise le lazy loading pour charger les pages à la demande. Cette approche améliore la performance initiale, car toutes les pages ne sont pas chargées au premier affichage.

5.3 Architecture backend

Le backend suit l'organisation standard de Laravel :

- routes/api.php : définit les routes de l'API.
- app/Http/Controllers/Api/ : contient les contrôleurs publics.
- app/Http/Controllers/Api/Admin/ : contient les contrôleurs réservés à l'administration.
- app/Models/ : contient les modèles Eloquent tels que Actualite, Programme, Publication, Statistique et Partenaire.
- database/migrations/ : contient la structure des tables.
- database/seeder/ : contient les données initiales.
- app/Http/Resources/ : permet de formater les réponses API.

Cette organisation permet de séparer clairement les responsabilités et de rendre le code plus lisible.

6. Réalisation de la partie backend

6.1 Modèles de données

Le projet contient plusieurs modèles représentant les contenus principaux du portail :

- Actualite : représente les nouvelles et annonces publiées par l'institution.
- Programme : représente les programmes ou initiatives présentés sur le site.
- Publication : représente les documents accessibles aux visiteurs.
- Statistique : représente les indicateurs chiffrés affichés sur le portail.
- Partenaire : représente les partenaires associés au ministère.
- User : représente les utilisateurs ayant accès à l'administration.

Ces modèles sont reliés à des migrations qui définissent les tables de la base de données.

6.2 API publique

L'API publique permet au frontend d'afficher les contenus aux visiteurs. Les principales routes sont :

- GET /api/v1/actualites
- GET /api/v1/actualites/{slug}
- GET /api/v1/programmes
- GET /api/v1/programmes/{slug}
- GET /api/v1/publications
- GET /api/v1/publications/{slug}
- GET /api/v1/statistiques
- POST /api/v1/contact

Ces routes permettent de récupérer les informations nécessaires à l'affichage des pages publiques et de traiter les messages envoyés via le formulaire de contact.

6.3 Authentification et espace administrateur

L'authentification est assurée avec Laravel Sanctum. L'administrateur peut se connecter via une route dédiée, puis accéder aux endpoints protégés par le middleware `auth:sanctum`.

Les routes d'administration permettent de gérer les actualités, les programmes, les publications et les statistiques. Elles sont regroupées sous le préfixe `/api/v1/admin`, ce qui facilite leur identification et leur protection.

6.4 Ressources API

Les ressources Laravel permettent de structurer les réponses envoyées au frontend. Elles assurent une sortie propre, cohérente et adaptée à l'affichage. Cette approche évite d'exposer directement toute la structure interne des modèles.

7. Réalisation de la partie frontend

7.1 Interface publique

L'interface publique comprend plusieurs pages :

- Accueil : présente les informations principales, les statistiques, les programmes, les actualités et les appels à l'action.
- Actualités : affiche la liste des actualités disponibles.
- Détail d'actualité : permet de lire une actualité complète.
- Programmes : présente les programmes de l'institution.
- Détail de programme : affiche les informations détaillées d'un programme.
- Publications : permet d'accéder aux documents disponibles.
- Contact : contient un formulaire permettant aux visiteurs d'envoyer un message.
- Ministère : présente l'institution et ses missions.

L'interface a été conçue pour être claire, responsive et agréable à utiliser.

7.2 Interface d'administration

L'interface d'administration comprend :

- une page de connexion ;
- un tableau de bord ;
- une gestion des actualités ;
- une gestion des programmes ;
- une gestion des publications ;
- une gestion des statistiques.

Les pages d'administration sont protégées par un composant `ProtectedRoute`, qui empêche l'accès aux utilisateurs non authentifiés. Cette logique améliore la sécurité de l'application et sépare clairement l'espace public de l'espace privé.

7.3 Gestion de la langue

Le projet intègre un contexte de langue permettant de gérer le français et l'arabe. Le contexte fournit notamment :

- la langue active ;
- la direction du texte, notamment le mode RTL pour l'arabe ;
- la persistance du choix de langue dans le stockage local ;
- une base pour une internationalisation plus complète avec `i18next`.

Cette fonctionnalité est importante dans un portail institutionnel destiné à un public large.

7.4 Hooks personnalisés

Plusieurs hooks personnalisés ont été mis en place :

- useApi : facilite les appels à l'API avec gestion du chargement, des erreurs et de l'annulation des requêtes.
- useScrollAnimation : déclenche des animations lorsqu'un élément entre dans le viewport.
- useMouseTilt : ajoute un effet visuel interactif au survol.

Ces hooks permettent de rendre le code plus réutilisable et plus propre.

8. Sécurité et gestion des accès

La sécurité est un point important du projet, surtout pour la partie administration. Les principales mesures prévues sont :

- authentification via Laravel Sanctum ;
- protection des routes admin par middleware ;
- séparation entre routes publiques et routes privées ;
- utilisation d'un contexte d'authentification côté frontend ;
- redirection des utilisateurs non connectés ;
- gestion de la déconnexion.

Même si le projet constitue une base fonctionnelle, certaines améliorations peuvent encore être ajoutées, comme la validation avancée des formulaires, la gestion fine des rôles ou l'audit des actions administrateur.

9. Tests et validation

La validation du projet peut être organisée selon plusieurs axes :

9.1 Validation fonctionnelle

Les fonctionnalités principales à vérifier sont :

- affichage correct de la page d'accueil ;
- navigation entre les pages publiques ;
- récupération des actualités depuis l'API ;
- affichage des détails par slug ;
- récupération des programmes et publications ;
- envoi du formulaire de contact ;
- connexion à l'espace administrateur ;
- accès protégé aux pages admin ;
- création, modification et suppression des contenus administrables.

9.2 Validation technique

Les points techniques à vérifier sont :

- compilation du frontend avec Vite ;
- démarrage du serveur Laravel ;
- exécution des migrations ;
- cohérence des réponses API ;
- fonctionnement de l'authentification Sanctum ;
- absence d'erreurs JavaScript dans le navigateur ;
- adaptation responsive sur mobile et desktop.

9.3 Tests automatisés

Le backend contient une structure de tests Laravel avec PHPUnit. Les tests présents peuvent être enrichis pour couvrir les contrôleurs API, l'authentification, la validation des formulaires et les opérations CRUD de l'administration.

10. Difficultés rencontrées et solutions

10.1 Structuration du projet full-stack

La première difficulté concernait l'organisation d'un projet séparant frontend et backend. Pour y répondre, le projet a été divisé en deux dossiers distincts : frontend pour React et backend pour Laravel. Cette séparation rend le projet plus lisible et facilite la maintenance.

10.2 Communication entre React et Laravel

La communication entre l'interface React et l'API Laravel nécessite une configuration correcte des URLs, des requêtes HTTP et des réponses JSON. L'utilisation d'Axios et d'un fichier utilitaire dédié permet de centraliser cette logique.

10.3 Protection de l'espace administrateur

L'accès à l'administration doit être limité aux utilisateurs authentifiés. Cette difficulté a été traitée avec Laravel Sanctum côté backend et un contexte d'authentification côté frontend.

10.4 Organisation des composants

Un portail institutionnel contient plusieurs pages et sections. Pour éviter la répétition du code, l'interface a été divisée en composants réutilisables : boutons, badges, sections, layout, navbar, footer et composants administrateur.

11. Apports du stage

Ce stage a permis au stagiaire de développer plusieurs compétences techniques et professionnelles.

11.1 Compétences techniques

Les connaissances techniques ont été renforcées dans les domaines suivants :

- développement d'interfaces avec React ;
- utilisation de Vite pour un environnement frontend moderne ;
- création d'un design responsive avec Tailwind CSS ;
- mise en place d'une API REST avec Laravel ;
- gestion des modèles, migrations et seeders ;
- authentification avec Laravel Sanctum ;
- intégration frontend-backend avec Axios ;
- organisation d'un projet full-stack ;
- utilisation de Git et GitHub.

11.2 Compétences professionnelles

Le stage a également contribué à améliorer :

- la capacité d'analyse des besoins ;
- l'autonomie dans la résolution de problèmes ;
- la rigueur dans l'organisation du code ;
- la communication avec les encadrants ;
- la capacité à documenter un projet ;
- la compréhension du cycle de développement d'une application web.

12. Perspectives d'amélioration

Le projet peut être amélioré avec plusieurs évolutions :

- ajout d'un système complet de rôles et permissions ;
- amélioration de la gestion des médias et fichiers ;
- ajout d'un moteur de recherche interne ;
- mise en place d'un tableau de bord statistique plus avancé ;
- enrichissement des tests automatisés ;
- optimisation SEO des pages publiques ;
- amélioration de l'accessibilité ;
- ajout d'une internationalisation complète français/arabe ;
- préparation d'un déploiement en production ;
- ajout d'un système de notifications ou de newsletters.

Ces perspectives permettraient de transformer la base actuelle en une plateforme institutionnelle plus complète et prête pour un usage réel.

Conclusion générale

Ce stage a été une expérience enrichissante qui a permis au stagiaire de participer à la réalisation d'un portail web institutionnel moderne. Le projet a offert l'occasion de travailler sur une architecture full-stack, en combinant React pour la partie frontend et Laravel pour la partie backend.

La réalisation du portail a nécessité plusieurs étapes : analyse des besoins, conception de l'architecture, développement des pages publiques, création de l'espace administrateur, intégration des API et prise en compte de la sécurité. Grâce à ce travail, les compétences techniques du stagiaire ont été consolidées, avec une meilleure compréhension des exigences d'un projet web professionnel.

Le résultat obtenu constitue une base solide pour un portail institutionnel évolutif. Il peut être enrichi progressivement avec de nouvelles fonctionnalités, une meilleure couverture de tests, une internationalisation complète et un déploiement en production.

En conclusion, ce stage a permis une progression à la fois technique et professionnelle. Il a donné une vision plus concrète du développement d'une application web complète et a préparé le stagiaire à travailler sur des projets plus complexes à l'avenir.

Bibliographie et ressources

- Documentation officielle React.
- Documentation officielle Laravel.
- Documentation officielle Tailwind CSS.
- Documentation officielle Vite.
- Documentation Laravel Sanctum.
- Ressources internes du projet : README, cahier des charges et code source.

Annexes

Annexe 1 : Structure simplifiée du projet

```
projet-stage/  
├── frontend/  
│   ├── src/  
│   │   ├── components/  
│   │   ├── context/  
│   │   ├── hooks/  
│   │   ├── pages/  
│   │   └── utils/  
│   └── package.json  
├── backend/  
│   ├── app/  
│   │   ├── Http/  
│   │   └── Models/  
│   ├── database/  
│   │   ├── migrations/  
│   │   └── seeders/  
│   ├── routes/  
│   └── composer.json  
└── cahier de charge.pdf
```

```
|— README.md
|— rapport-de-stage.md
```

Annexe 2 : Principales routes publiques

```
GET /api/v1/actualites
GET /api/v1/actualites/{slug}
GET /api/v1/programmes
GET /api/v1/programmes/{slug}
GET /api/v1/publications
GET /api/v1/publications/{slug}
GET /api/v1/statistiques
POST /api/v1/contact
```

Annexe 3 : Principales routes d'administration

```
POST /api/v1/auth/login
POST /api/v1/admin/auth/logout
GET /api/v1/admin/auth/me
CRUD /api/v1/admin/actualites
CRUD /api/v1/admin/programmes
CRUD /api/v1/admin/publications
GET /api/v1/admin/statistiques
PUT /api/v1/admin/statistiques/{id}
```

Annexe 4 : Vérification avant dépôt final

- Relire le rapport et corriger les éventuelles fautes de forme.
- Ajouter des captures d'écran de l'application si elles sont demandées par l'établissement.
- Vérifier la mise en page après export en PDF ou DOCX.
- Ajouter les signatures ou validations administratives si elles sont exigées.